

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

C:\Users\Shaki\anaconda3\Lib\site-packages\pandas\core\arrays\masked.py:60: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
from pandas.core import (

```
In [2]: # Load dataset
df = pd.read_csv('household_power_consumption.txt',
                 sep=';',
                 na_values=['?'],
                 low_memory=False)
```

```
In [3]: # Basic exploration
print("Shape:", df.shape)
print("\nDtypes:\n", df.dtypes)
print("\nMissing values:\n", df.isnull().sum())
print("\nBasic Stats:\n", df.describe())
print("\nFirst 5 rows:\n", df.head())
```

Shape: (2075259, 9)

Dtypes:

Date	object
Time	object
Global_active_power	float64
Global_reactive_power	float64
Voltage	float64
Global_intensity	float64
Sub_metering_1	float64
Sub_metering_2	float64
Sub_metering_3	float64

dtype: object

Missing values:

Date	0
Time	0
Global_active_power	25979
Global_reactive_power	25979
Voltage	25979
Global_intensity	25979
Sub_metering_1	25979
Sub_metering_2	25979
Sub_metering_3	25979

dtype: int64

Basic Stats:

	Global_active_power	Global_reactive_power	Voltage \
count	2.049280e+06	2.049280e+06	2.049280e+06
mean	1.091615e+00	1.237145e-01	2.408399e+02
std	1.057294e+00	1.127220e-01	3.239987e+00
min	7.600000e-02	0.000000e+00	2.232000e+02
25%	3.080000e-01	4.800000e-02	2.389900e+02
50%	6.020000e-01	1.000000e-01	2.410100e+02
75%	1.528000e+00	1.940000e-01	2.428900e+02
max	1.112200e+01	1.390000e+00	2.541500e+02

	Global_intensity	Sub_metering_1	Sub_metering_2	Sub_metering_3
count	2.049280e+06	2.049280e+06	2.049280e+06	2.049280e+06
mean	4.627759e+00	1.121923e+00	1.298520e+00	6.458447e+00
std	4.444396e+00	6.153031e+00	5.822026e+00	8.437154e+00
min	2.000000e-01	0.000000e+00	0.000000e+00	0.000000e+00
25%	1.400000e+00	0.000000e+00	0.000000e+00	0.000000e+00
50%	2.600000e+00	0.000000e+00	0.000000e+00	1.000000e+00
75%	6.400000e+00	0.000000e+00	1.000000e+00	1.700000e+01
max	4.840000e+01	8.800000e+01	8.000000e+01	3.100000e+01

First 5 rows:

	Date	Time	Global_active_power	Global_reactive_power	Voltage
0	16/12/2006	17:24:00	4.216	0.418	234.84
1	16/12/2006	17:25:00	5.360	0.436	233.63
2	16/12/2006	17:26:00	5.374	0.498	233.29
3	16/12/2006	17:27:00	5.388	0.502	233.74
4	16/12/2006	17:28:00	3.666	0.528	235.68

Global_intensity	Sub_metering_1	Sub_metering_2	Sub_metering_3
------------------	----------------	----------------	----------------

0	18.4	0.0	1.0	17.0
1	23.0	0.0	1.0	16.0
2	23.0	0.0	2.0	17.0
3	23.0	0.0	1.0	17.0
4	15.8	0.0	1.0	17.0

```
In [4]: # Combine Date + Time into single datetime column
df['Datetime'] = pd.to_datetime(df['Date'] + ' ' + df['Time'], dayfirst=True)
df.drop(columns=['Date', 'Time'], inplace=True)
df.set_index('Datetime', inplace=True)

# Cast all columns to float
df = df.astype(float)

print("Missing before imputation:\n", df.isnull().sum())

# Linear interpolation for missing values
df.interpolate(method='linear', inplace=True)

print("\nMissing after imputation:\n", df.isnull().sum())
print("\nShape after cleaning:", df.shape)
```

Missing before imputation:

Global_active_power	25979
Global_reactive_power	25979
Voltage	25979
Global_intensity	25979
Sub_metering_1	25979
Sub_metering_2	25979
Sub_metering_3	25979

dtype: int64

Missing after imputation:

Global_active_power	0
Global_reactive_power	0
Voltage	0
Global_intensity	0
Sub_metering_1	0
Sub_metering_2	0
Sub_metering_3	0

dtype: int64

Shape after cleaning: (2075259, 7)

```
In [5]: # Resample to 30-minute frequency
df_30min = df.resample('30T').mean()

print("Shape after resampling:", df_30min.shape)
print(df_30min.head())

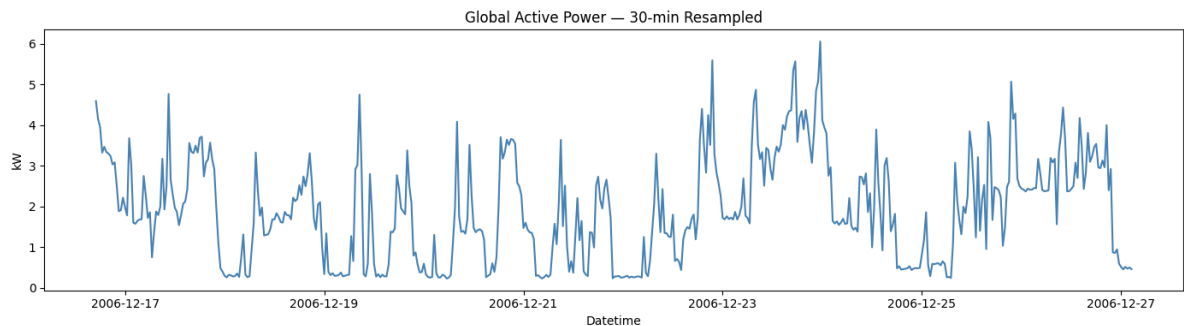
# Plot to verify smoothing
plt.figure(figsize=(14, 4))
plt.plot(df_30min['Global_active_power'][:500], color='steelblue')
plt.title('Global Active Power - 30-min Resampled')
plt.xlabel('Datetime')
plt.ylabel('kW')
plt.tight_layout()
plt.show()
```

Shape after resampling: (69177, 7)

	Global_active_power	Global_reactive_power	Voltage
Datetime			
2006-12-16 17:00:00	4.587333	0.484000	234.366667
2006-12-16 17:30:00	4.150000	0.178000	234.699333
2006-12-16 18:00:00	3.944800	0.084867	235.184667
2006-12-16 18:30:00	3.319600	0.075200	233.975667
2006-12-16 19:00:00	3.464400	0.049800	233.754000

	Global_intensity	Sub_metering_1	Sub_metering_2	
Datetime				
2006-12-16 17:00:00	19.700000	0.0	1.333333	
2006-12-16 17:30:00	17.780000	0.0	0.366667	
2006-12-16 18:00:00	16.966667	0.0	11.400000	
2006-12-16 18:30:00	14.233333	0.0	2.033333	
2006-12-16 19:00:00	14.746667	0.0	1.833333	

	Sub_metering_3
Datetime	
2006-12-16 17:00:00	16.833333
2006-12-16 17:30:00	16.866667
2006-12-16 18:00:00	16.966667
2006-12-16 18:30:00	16.766667
2006-12-16 19:00:00	16.766667



```

In [6]: # Work on 30-min resampled data
df_feat = df_30min.copy()

# Datetime features
df_feat['hour'] = df_feat.index.hour
df_feat['dayofweek'] = df_feat.index.dayofweek
df_feat['month'] = df_feat.index.month
df_feat['quarter'] = df_feat.index.quarter
df_feat['is_weekend'] = (df_feat.index.dayofweek >= 5).astype(int)

# Lag features (30-min lag = 1 step back, 1-hour lag = 2 steps back)
df_feat['lag_1'] = df_feat['Global_active_power'].shift(1)
df_feat['lag_2'] = df_feat['Global_active_power'].shift(2)

# Rolling mean (last 2 hours = 4 steps of 30-min)
df_feat['rolling_mean_4'] = df_feat['Global_active_power'].shift(1).rolling(4)

# Drop NaN rows created by lag/rolling
df_feat.dropna(inplace=True)

# Final 9 features
features = ['hour', 'dayofweek', 'month', 'quarter', 'is_weekend',
            'lag_1', 'lag_2', 'rolling_mean_4',
            'Global_voltage'] # 9th feature - electrical signal

print("Feature columns:", features)
print("Shape:", df_feat.shape)

```

```

Feature columns: ['hour', 'dayofweek', 'month', 'quarter', 'is_weekend', 'lag_1', 'lag_2', 'rolling_mean_4', 'Global_voltage']
Shape: (69173, 15)

```

```

In [7]: from statsmodels.tsa.arima.model import ARIMA
        from sklearn.metrics import mean_squared_error

        # Use last 18K records of 30-min resampled data
        series = df_30min['Global_active_power'].dropna()
        series = series[-18000:]

        # Train/test split - 80/20
        split = int(len(series) * 0.8)
        train_arima = series[:split]
        test_arima = series[split:]

        print(f"ARIMA Train: {len(train_arima)} | Test: {len(test_arima)}")

        # Fit ARIMA(5,1,0)
        model_arima = ARIMA(train_arima, order=(5, 1, 0))
        model_arima_fit = model_arima.fit()

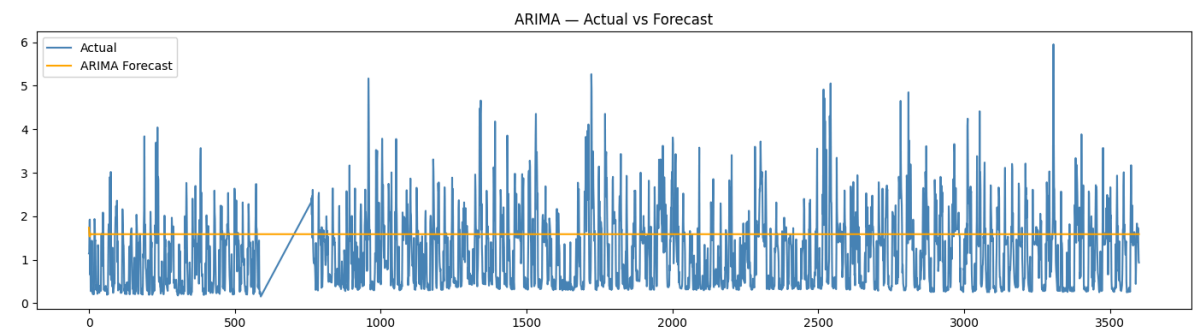
        # Forecast
        forecast_arima = model_arima_fit.forecast(steps=len(test_arima))

        # RMSE
        rmse_arima = np.sqrt(mean_squared_error(test_arima, forecast_arima))
        print(f"ARIMA RMSE: {rmse_arima:.4f}")

        # Plot
        plt.figure(figsize=(14, 4))
        plt.plot(test_arima.values, label='Actual', color='steelblue')
        plt.plot(forecast_arima.values, label='ARIMA Forecast', color='orange')
        plt.title('ARIMA - Actual vs Forecast')
        plt.legend()
        plt.tight_layout()
        plt.show()

```

ARIMA Train: 14400 | Test: 3600
 ARIMA RMSE: 0.9616



```

In [11]: from xgboost import XGBRegressor
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import mean_squared_error

# Features and target
X = df_feat[features]
y = df_feat['Global_active_power']

# Train/test split - no shuffle (time-series order matters)
split = int(len(X) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

print(f"XGBoost Train: {len(X_train)} | Test: {len(X_test)}")

# Grid search
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.05, 0.1],
    'subsample': [0.8, 1.0]
}

xgb = XGBRegressor(random_state=42, verbosity=0)
grid_search = GridSearchCV(xgb, param_grid, cv=3,
                           scoring='neg_root_mean_squared_error',
                           n_jobs=-1, verbose=1)
grid_search.fit(X_train, y_train)

print("Best params:", grid_search.best_params_)

# Best model
best_xgb = grid_search.best_estimator_
y_pred_xgb = best_xgb.predict(X_test)

# RMSE
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))
print(f"XGBoost RMSE: {rmse_xgb:.4f}")

# Plot
plt.figure(figsize=(14, 4))
plt.plot(y_test.values[:500], label='Actual', color='steelblue')
plt.plot(y_pred_xgb[:500], label='XGBoost Forecast', color='green')
plt.title('XGBoost - Actual vs Predicted')
plt.legend()
plt.tight_layout()
plt.show()

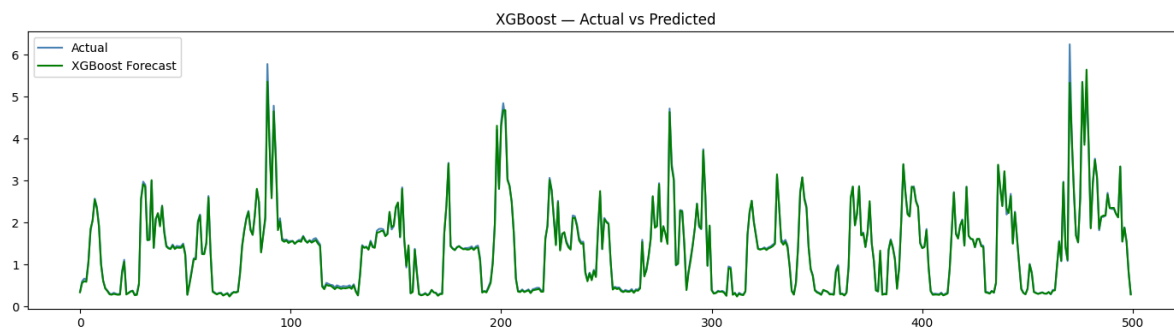
```

XGBoost Train: 55338 | Test: 13835

Fitting 3 folds for each of 16 candidates, totalling 48 fits

Best params: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 200, 'subsample': 0.8}

XGBoost RMSE: 0.0340




```

In [12]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error

# 4 input features for multivariate LSTM
lstm_features = ['Global_active_power', 'Sub_metering_1',
                 'Sub_metering_2', 'Sub_metering_3']

lstm_data = df_30min[lstm_features].dropna()
lstm_data = lstm_data[-18000:] # latest 18K records

# Scale
scaler = MinMaxScaler()
scaled = scaler.fit_transform(lstm_data)

# Create sequences - 24 timesteps to predict next value
def create_sequences(data, seq_len=24):
    X, y = [], []
    for i in range(len(data) - seq_len):
        X.append(data[i:i+seq_len]) # all 4 features
        y.append(data[i+seq_len][0]) # only Global_active_power
    return np.array(X), np.array(y)

SEQ_LEN = 24
X_lstm, y_lstm = create_sequences(scaled, SEQ_LEN)

# Train/test split
split = int(len(X_lstm) * 0.8)
X_train_1, X_test_1 = X_lstm[:split], X_lstm[split:]
y_train_1, y_test_1 = y_lstm[:split], y_lstm[split:]

print(f"LSTM Train: {X_train_1.shape} | Test: {X_test_1.shape}")

# Build LSTM model
model_lstm = Sequential([
    LSTM(64, return_sequences=True, input_shape=(SEQ_LEN, 4)),
    Dropout(0.2),
    LSTM(32),
    Dropout(0.2),
    Dense(1)
])

model_lstm.compile(optimizer='adam', loss='mse')
model_lstm.summary()

# Train
history = model_lstm.fit(X_train_1, y_train_1,
                        epochs=10,
                        batch_size=64,
                        validation_split=0.1,
                        verbose=1)

# Predict
y_pred_lstm_scaled = model_lstm.predict(X_test_1)

```

```

# Inverse transform - only Global_active_power column
def inverse_target(scaled_vals, scaler, n_features=4):
    dummy = np.zeros((len(scaled_vals), n_features))
    dummy[:, 0] = scaled_vals.flatten()
    return scaler.inverse_transform(dummy[:, 0])

y_pred_lstm = inverse_target(y_pred_lstm_scaled, scaler)
y_test_lstm = inverse_target(y_test_l, scaler)

# RMSE
rmse_lstm = np.sqrt(mean_squared_error(y_test_lstm, y_pred_lstm))
print(f"LSTM RMSE: {rmse_lstm:.4f}")

# Plot
plt.figure(figsize=(14, 4))
plt.plot(y_test_lstm[:500], label='Actual', color='steelblue')
plt.plot(y_pred_lstm[:500], label='LSTM Forecast', color='red')
plt.title('LSTM - Actual vs Predicted')
plt.legend()
plt.tight_layout()
plt.show()

```

WARNING:tensorflow:From C:\Users\Shaki\anaconda3\Lib\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

LSTM Train: (14380, 24, 4) | Test: (3596, 24, 4)

WARNING:tensorflow:From C:\Users\Shaki\anaconda3\Lib\site-packages\keras\src\layers\rnn\lstm.py:148: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

WARNING:tensorflow:From C:\Users\Shaki\anaconda3\Lib\site-packages\keras\src\optimizers_init_.py:309: The name tf.train.Optimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 24, 64)	17664
dropout (Dropout)	(None, 24, 64)	0
lstm_1 (LSTM)	(None, 32)	12416
dropout_1 (Dropout)	(None, 32)	0
dense (Dense)	(None, 1)	33

Total params: 30113 (117.63 KB)
Trainable params: 30113 (117.63 KB)
Non-trainable params: 0 (0.00 Byte)

Epoch 1/10

WARNING:tensorflow:From C:\Users\Shaki\anaconda3\Lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.RaggedTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

203/203 [=====] - 42s 94ms/step - loss: 0.0132 - val_loss: 0.0060

Epoch 2/10

203/203 [=====] - 13s 63ms/step - loss: 0.0097 - val_loss: 0.0052

Epoch 3/10

203/203 [=====] - 13s 66ms/step - loss: 0.0084 - val_loss: 0.0046

Epoch 4/10

203/203 [=====] - 12s 59ms/step - loss: 0.0080 - val_loss: 0.0045

Epoch 5/10

203/203 [=====] - 13s 63ms/step - loss: 0.0078 - val_loss: 0.0044

Epoch 6/10

203/203 [=====] - 13s 63ms/step - loss: 0.0075 - val_loss: 0.0043

Epoch 7/10

203/203 [=====] - 11s 56ms/step - loss: 0.0073 - val

_loss: 0.0044

Epoch 8/10

203/203 [=====] - 11s 55ms/step - loss: 0.0073 - val_

_loss: 0.0043

Epoch 9/10

203/203 [=====] - 8s 40ms/step - loss: 0.0072 - val_

loss: 0.0043

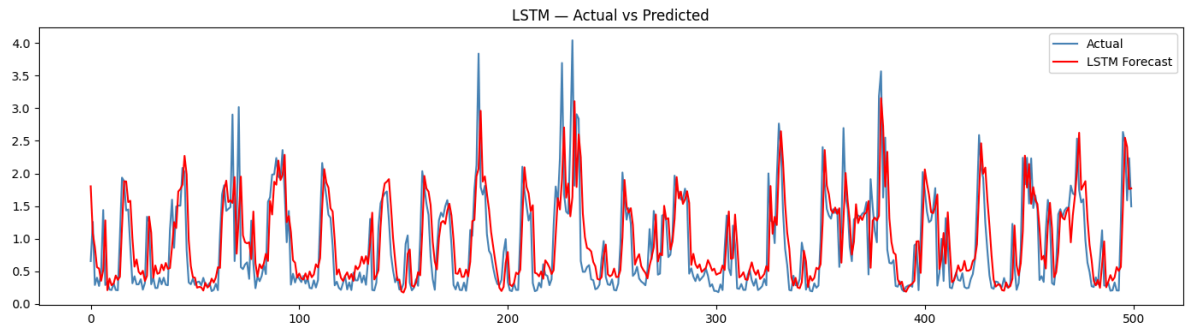
Epoch 10/10

203/203 [=====] - 10s 47ms/step - loss: 0.0071 - val_

_loss: 0.0044

113/113 [=====] - 3s 10ms/step

LSTM RMSE: 0.5295



```

In [13]: # RMSE Comparison Table
results = pd.DataFrame({
    'Model': ['ARIMA', 'XGBoost', 'LSTM'],
    'RMSE': [round(rmse_arima, 4), round(rmse_xgb, 4), round(rmse_lstm, 4)]
})
print(results.to_string(index=False))

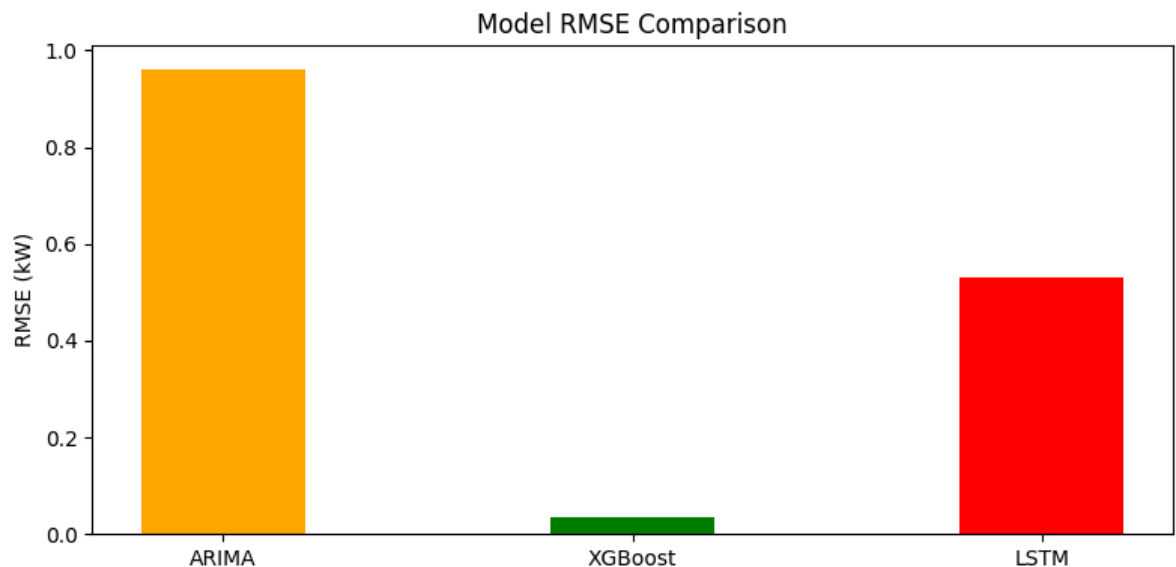
# Bar chart
plt.figure(figsize=(8, 4))
colors = ['orange', 'green', 'red']
plt.bar(results['Model'], results['RMSE'], color=colors, width=0.4)
plt.title('Model RMSE Comparison')
plt.ylabel('RMSE (kW)')
plt.tight_layout()
plt.show()

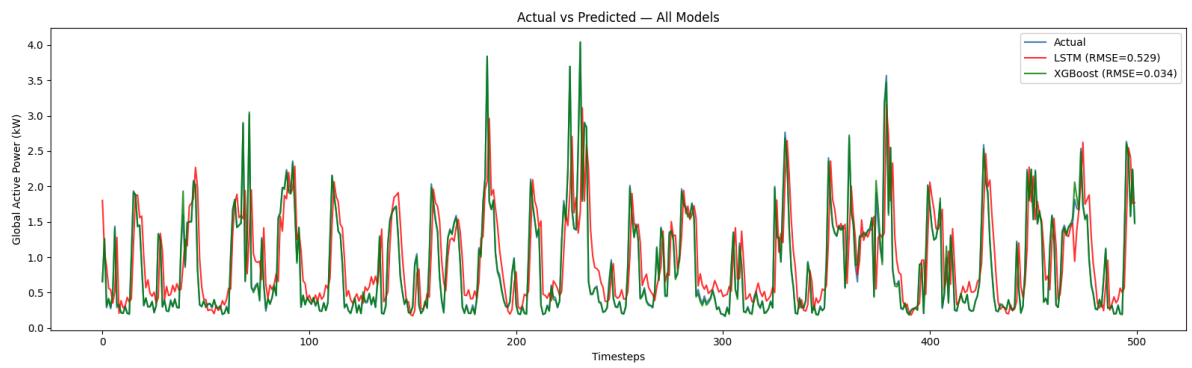
# Overlay plot - Actual vs all 3 models (first 500 test points)
n = 500
actual_plot = y_test_lstm[:n] # common reference - LSTM test set

plt.figure(figsize=(16, 5))
plt.plot(actual_plot, label='Actual', color='steelblue', linewidth=1.5)
plt.plot(y_pred_lstm[:n], label=f'LSTM (RMSE={rmse_lstm:.3f})', color='red', a
plt.plot(y_pred_xgb[-len(y_test_lstm):][:n], label=f'XGBoost (RMSE={rmse_xgb:.
plt.title('Actual vs Predicted - All Models')
plt.xlabel('Timesteps')
plt.ylabel('Global Active Power (kW)')
plt.legend()
plt.tight_layout()
plt.show()

```

Model	RMSE
ARIMA	0.9616
XGBoost	0.0340
LSTM	0.5295





In []:

In []:

In []: